



INDUSTRIAL IOT AND INDUSTRIAL 4.0

SMART AGRICULTURE(IRRIGATION)

Name	GOKUL D
Reg Number	511323106019
College Code	5113
College Name	KINGSTON ENGINEERING COLLEGE
Department	Electronics and Communication Engineering

Aim

To design and develop an IoT-based Smart Irrigation System that automatically monitors soil moisture, temperature, and environmental conditions in real-time, and efficiently controls water flow to crops using intelligent decision-making, thereby optimizing water usage, improving crop yield, and promoting sustainable agriculture.

Introduction

Agriculture is the backbone of many economies, and water management plays a crucial role in maximizing crop yield. Traditional irrigation methods often lead to over-irrigation or under-irrigation, causing reduced crop productivity and wastage of water. With the rise of Internet of Things (IoT) technologies, smart irrigation systems have emerged as a modern solution to address these challenges.

Concept

IoT-enabled smart irrigation uses sensors, controllers, cloud computing, and communication networks to monitor soil and environmental conditions in real-time. Based on the collected data, the system automatically controls water distribution, ensuring that crops receive the optimal amount of water.

Components of IoT-based Smart Irrigation System

Sensors:

- Soil Moisture Sensor – monitors the water content in the soil.
- Temperature & Humidity Sensor – tracks climatic conditions.
- Water Flow Sensor – measures the volume of water used.
- Light/LDR Sensor – detects sunlight intensity for crop growth analysis.

Microcontroller / IoT Device:

- ESP32, Arduino, or Raspberry Pi is used to process sensor data.

Communication Module:

- Wi-Fi, GSM, or LoRa for data transmission to the cloud or mobile app.

Actuators:

- Solenoid valves or pumps controlled automatically to irrigate the fields.

Software Used

1. **Arduino IDE** – Used for writing, compiling, and uploading the ESP32 program.
2. **Wokwi Simulator** – Used for simulating the entire IoT circuit (ESP32, sensors, and relay).
3. **ThingSpeak Cloud Platform** – Used for real-time data storage, monitoring, and visualization of sensor data.
4. **MATLAB (ThingSpeak Integration)** – Used for data analysis and visualization of collected IoT data.
5. **GitHub** – Used for project version control and sharing the source code.
6. **Google Colab** – Used for performing cloud-based Python programming, data processing, and machine learning model training.
7. **Unity** – Used for creating 3D visualizations or virtual environments for IoT data representation.
8. **Power BI** – Used for interactive dashboards and advanced data visualization of IoT analytics.

Program :

Wokwi : <https://wokwi.com/projects/444958426812314625>

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <DHT.h>

// ===== WiFi & ThingSpeak Details =====
const char* ssid = "Wokwi-GUEST";    // Wokwi virtual network
const char* password = "";

const char* apiKey = "ZFM142K29N2VDA21"; // ThingSpeak Write API Key
const char* server = "http://api.thingspeak.com/update";
const String channelID = "3120758"; // Removed leading space

// ===== Sensor Pins =====
#define DHTPIN 4
#define DHTTYPE DHT22
#define SOIL_PIN 34
#define RELAY_PIN 26

// ===== Threshold Values =====
float moistureThreshold = 40.0; // (%) Pump ON if below this
float tempThreshold = 35.0;    // (°C) Pump ON if above this

// ===== Soil Calibration Values =====
```

```

int soilDry = 3500; // ADC value when dry
int soilWet = 1500; // ADC value when wet

// ===== Relay Setup =====
bool RELAY_ACTIVE_LOW = true;
bool pumpStatus = false;

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, RELAY_ACTIVE_LOW ? HIGH : LOW); // Pump OFF initially

  Serial.println("Connecting to WiFi...");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\n 🟢 WiFi Connected!");
}

float readSoilMoisture() {
  int rawValue = analogRead(SOIL_PIN);
  float percent = (float)(soilDry - rawValue) / (soilDry - soilWet) * 100.0;
  percent = constrain(percent, 0, 100);
  return percent;
}

```

```

void setPump(bool on) {
    digitalWrite(RELAY_PIN, RELAY_ACTIVE_LOW ? (on ? LOW : HIGH) : (on ? HIGH : LOW));
    if (on != pumpStatus) {
        pumpStatus = on;
        if (on)
            Serial.println("☁️ Pump is ON (Dry soil or high temperature detected)");
        else
            Serial.println("💧 Pump is OFF (Conditions normal)");
    }
}

```

```

void loop() {
    float soilMoisture = readSoilMoisture();
    float temperature = dht.readTemperature();

    bool turnPumpOn = false;

    if (soilMoisture < moistureThreshold || temperature > tempThreshold) {
        turnPumpOn = true;
    } else {
        turnPumpOn = false;
    }
}

```

```

setPump(turnPumpOn);

```

```

Serial.printf("🌱 Soil Moisture: %.1f%% | 🌡 Temp: %.1f°C | Pump: %s\n",
    soilMoisture, temperature, turnPumpOn ? "ON" : "OFF");

```

```

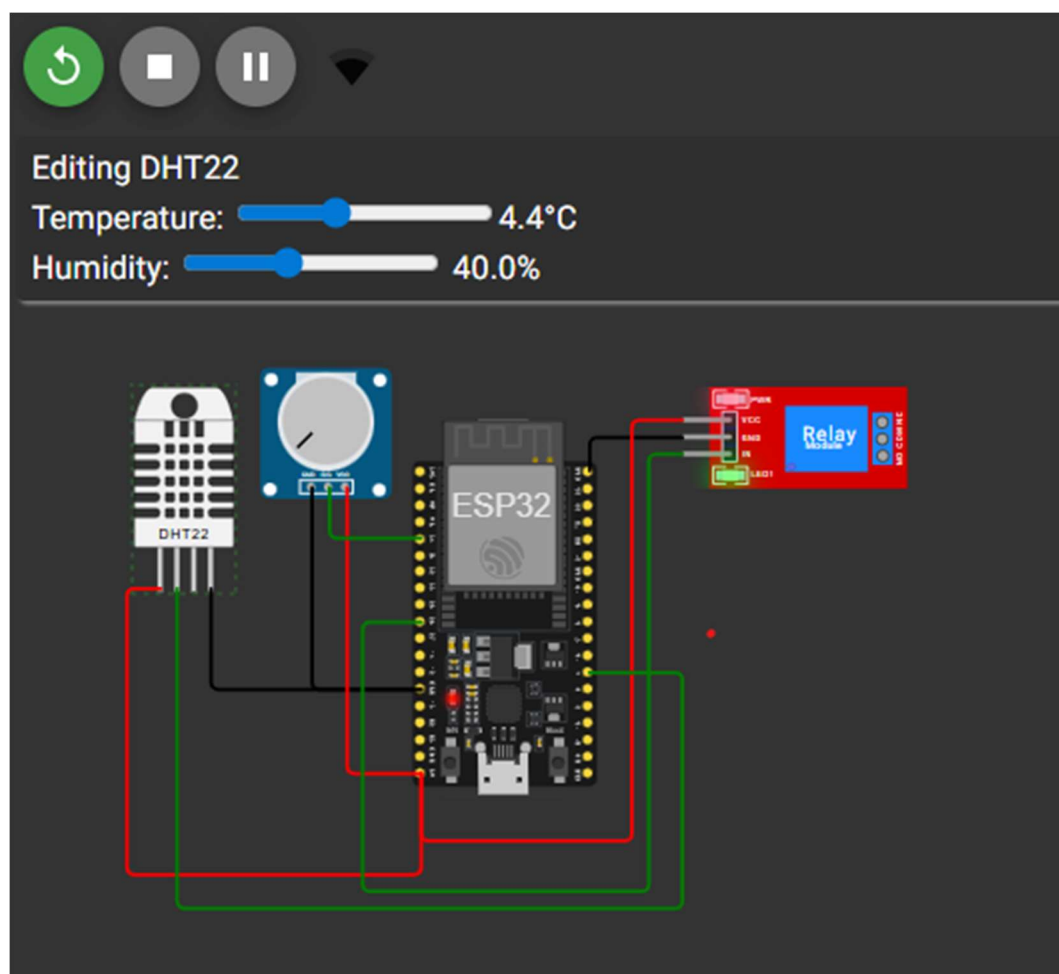
// Send data to ThingSpeak
if (WiFi.status() == WL_CONNECTED) {
  HTTPClient http;

  String url = String(server) +
    "?api_key=" + apiKey +
    "&field1=" + String(soilMoisture, 1) +
    "&field2=" + String(temperature, 1) +
    "&field3=" + String(turnPumpOn ? 1 : 0);

  http.begin(url);
  int httpCode = http.GET();
  if (httpCode > 0) {
    Serial.printf("📡 ThingSpeak Response: %d\n", httpCode);
  } else {
    Serial.printf("❌ Error sending data: %s\n", http.errorToString(httpCode).c_str());
  }
  http.end();
} else {
  Serial.println("⚠️ WiFi not connected!");
}

delay(20000); // Update every 20 seconds
}

```



entry 0x400805dc

Connecting to WiFi...

...

✓ WiFi Connected!

🌱 Soil Moisture: 100.0% | 🌡 Temp: 24.0°C | 🚰 Pump: OFF

📡 ThingSpeak Response: 200

🌱 Soil Moisture: 100.0% | 🌡 Temp: 24.0°C | 🚰 Pump: OFF

📡 ThingSpeak Response: 200

🌱 Soil Moisture: 100.0% | 🌡 Temp: 4.4°C | 🚰 Pump: OFF

📡 ThingSpeak Response: 200

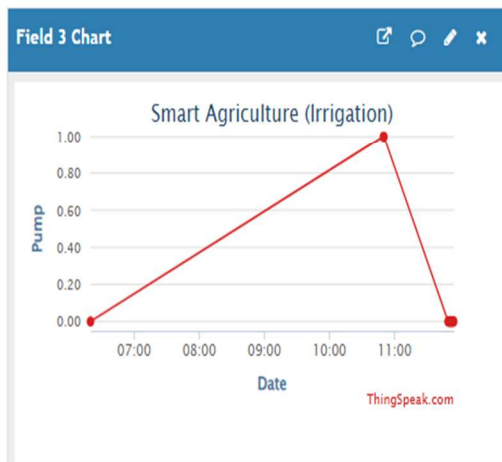
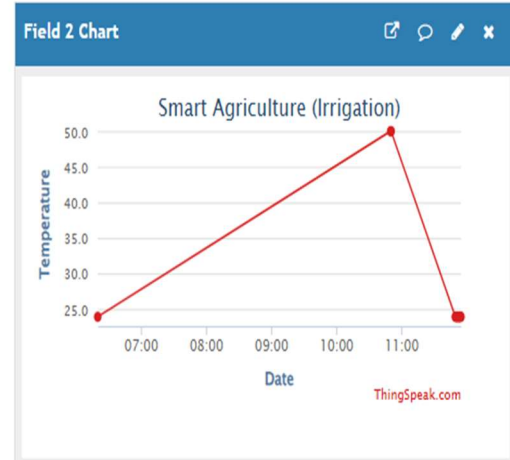
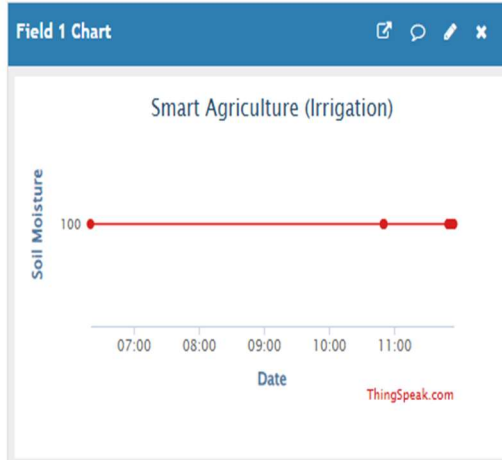
🌱 Soil Moisture: 100.0% | 🌡 Temp: 4.4°C | 🚰 Pump: OFF

📡 ThingSpeak Response: 200

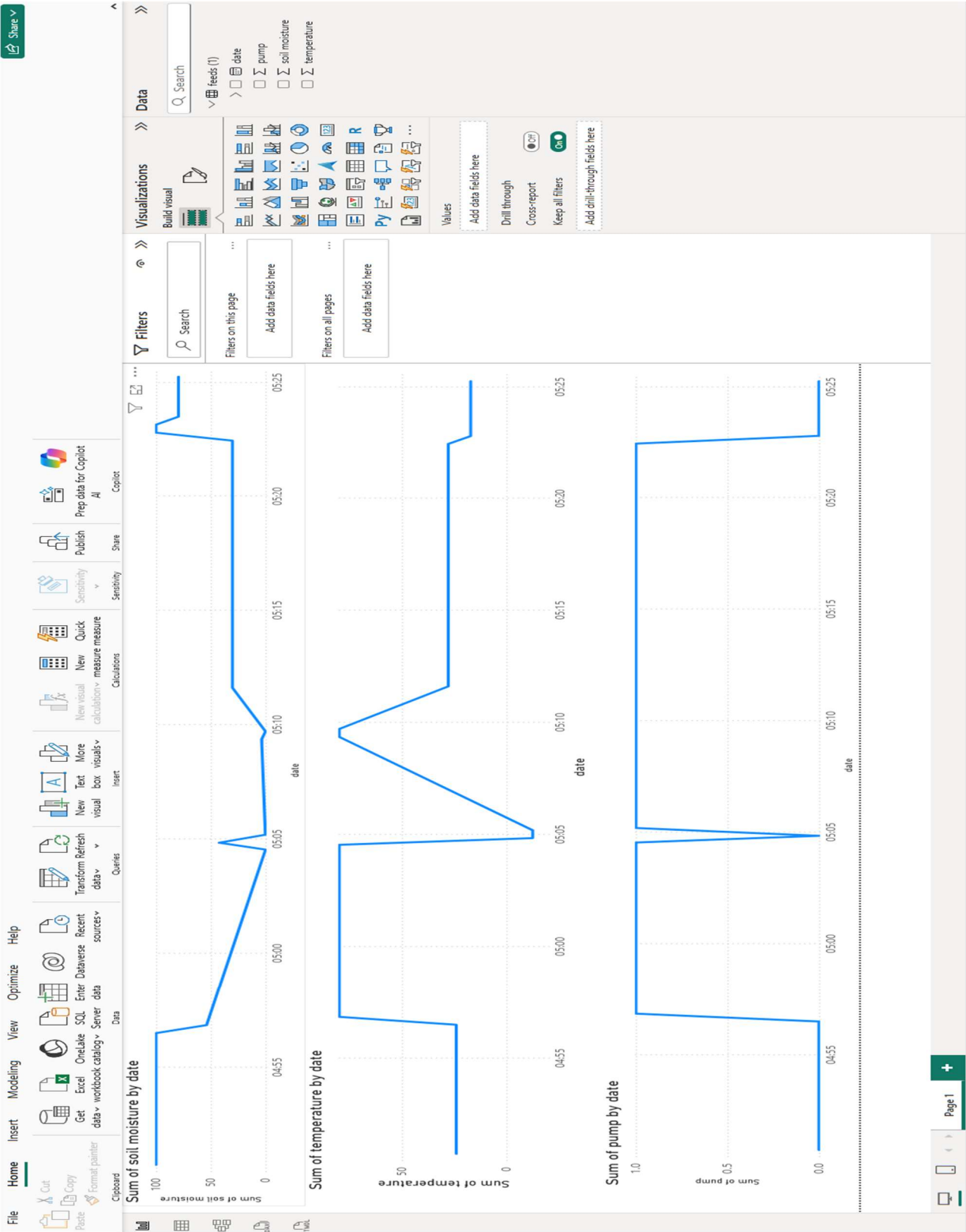
🌱 Soil Moisture: 100.0% | 🌡 Temp: 4.4°C | 🚰 Pump: OFF

📡 ThingSpeak Response: 200

Thingspeak: https://thingspeak.mathworks.com/channels/3119564/private_show



Power bi



Data

my project File Edit View Insert Format Data Tools Extensions Help Ask Gemini Q Menus ↶ ↷ ↵ ↻ 100% £ % .0 .00 123 Default...						
F16						
	A	B	C	D	E	F
1	date/time	moisture	temperature	pump		
2	16/10/2025 14:2	100	24	0		
3	16/10/2025 14:2	100	24	0		
4	16/10/2025 14:3	100	24	0		
5	16/10/2025 14:3	100	24	0		
6	16/10/2025 14:3	100	24	0		
7	16/10/2025 14:3	100	24	0		
8	16/10/2025 14:3	100	24	0		
9	16/10/2025 14:3	100	24	0		
10	16/10/2025 14:3	100	24	0		
11	16/10/2025 14:3	100	24	0		
12	16/10/2025 14:3	53.7	80	1		
13	16/10/2025 14:3	42.3	-12.6	0		
14	16/10/2025 14:3	42.3	-12.6	0		
15	16/10/2025 14:3	42.3	-12.6	0		
16	16/10/2025 14:3	42.3	-12.6	0		
17	16/10/2025 14:3	42.3	-12.6	0		
18	16/10/2025 14:3	42.3	-12.6	0		
19	16/10/2025 14:3	42.3	-12.6	0		
20	16/10/2025 14:3	42.3	-12.6	0		
21	16/10/2025 14:3	0	-12.6	1		
22	16/10/2025 14:4	3.6	80	1		
23	16/10/2025 14:4	3.6	80	1		
24	16/10/2025 14:4	3.6	80	1		
25	16/10/2025 14:4	0	80	1		
26	16/10/2025 14:4	0	80	1		
27	16/10/2025 14:4	0	80	1		
28	16/10/2025 14:4	0	80	1		
29	16/10/2025 14:4	0	80	1		
30	16/10/2025 14:4	30.1	27.8	1		
31	16/10/2025 14:4	30.1	27.8	1		
32	16/10/2025 14:4	30.1	27.8	1		
33	16/10/2025 14:4	30.1	27.8	1		
34	16/10/2025 14:4	30.1	27.8	1		
35	16/10/2025 14:4	30.1	27.8	1		
36	16/10/2025 14:4	30.1	27.8	1		
37	16/10/2025 14:4	30.1	27.8	1		
38	16/10/2025 14:4	30.1	27.8	1		
39	16/10/2025 14:4	30.1	27.8	1		
40	16/10/2025 14:4	30.1	27.8	1		
41	16/10/2025 14:4	30.1	27.8	1		
42	16/10/2025 14:5	30.1	27.8	1		
43	16/10/2025 14:5	30.1	27.8	1		
44	16/10/2025 14:5	30.1	27.8	1		
45	16/10/2025 14:5	30.1	27.8	1		

Google Colab

Program:

```
# 📖 Import Required Libraries
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import requests
```

```
from datetime import datetime
```

```
# 🔗 ThingSpeak READ API URL
```

```
url =
```

```
"https://api.thingspeak.com/channels/3119564/feeds.json?api_key=CQASS9XGW  
1J2MWPA&results=20"
```

```
# 📡 Fetch Data from ThingSpeak
```

```
response = requests.get(url)
```

```
data = response.json()
```

```
# ✅ Convert to DataFrame
```

```
feeds = data['feeds']
```

```
df = pd.DataFrame(feeds)
```

```
# ⌚ Convert 'created_at' to readable datetime format
```

```
df['created_at'] = pd.to_datetime(df['created_at'])
```

```
# 🔍 Extract fields (update field numbers if needed)
```

```
df['Soil_Moisture'] = df['field1'].astype(float)
```

```
df['Temperature'] = df['field2'].astype(float)
df['Pump_Status'] = df['field3'].astype(float)
```

```
# 📋 Display the latest data
```

```
print("✅ Latest Smart Irrigation Data:")
print(df[['created_at', 'Soil_Moisture', 'Temperature', 'Pump_Status']].tail())
```

```
# 📊 Plot Graphs
```

```
plt.figure(figsize=(12, 8))
```

```
# --- Soil Moisture ---
```

```
plt.subplot(3, 1, 1)
plt.plot(df['created_at'], df['Soil_Moisture'], color='green', marker='o')
plt.title('🌱 Soil Moisture Over Time')
plt.ylabel('Moisture (%)')
plt.grid(True)
```

```
# --- Temperature ---
```

```
plt.subplot(3, 1, 2)
plt.plot(df['created_at'], df['Temperature'], color='orange', marker='o')
plt.title('🌡️ Temperature Over Time')
plt.ylabel('Temperature (°C)')
plt.grid(True)
```

```
# --- Pump Status ---
```

```
plt.subplot(3, 1, 3)
```

```
plt.step(df['created_at'], df['Pump_Status'], where='post', color='blue')
```

```
plt.title('🚰 Pump Status Over Time')
```

```
plt.ylabel('Pump (1=ON, 0=OFF)')
```

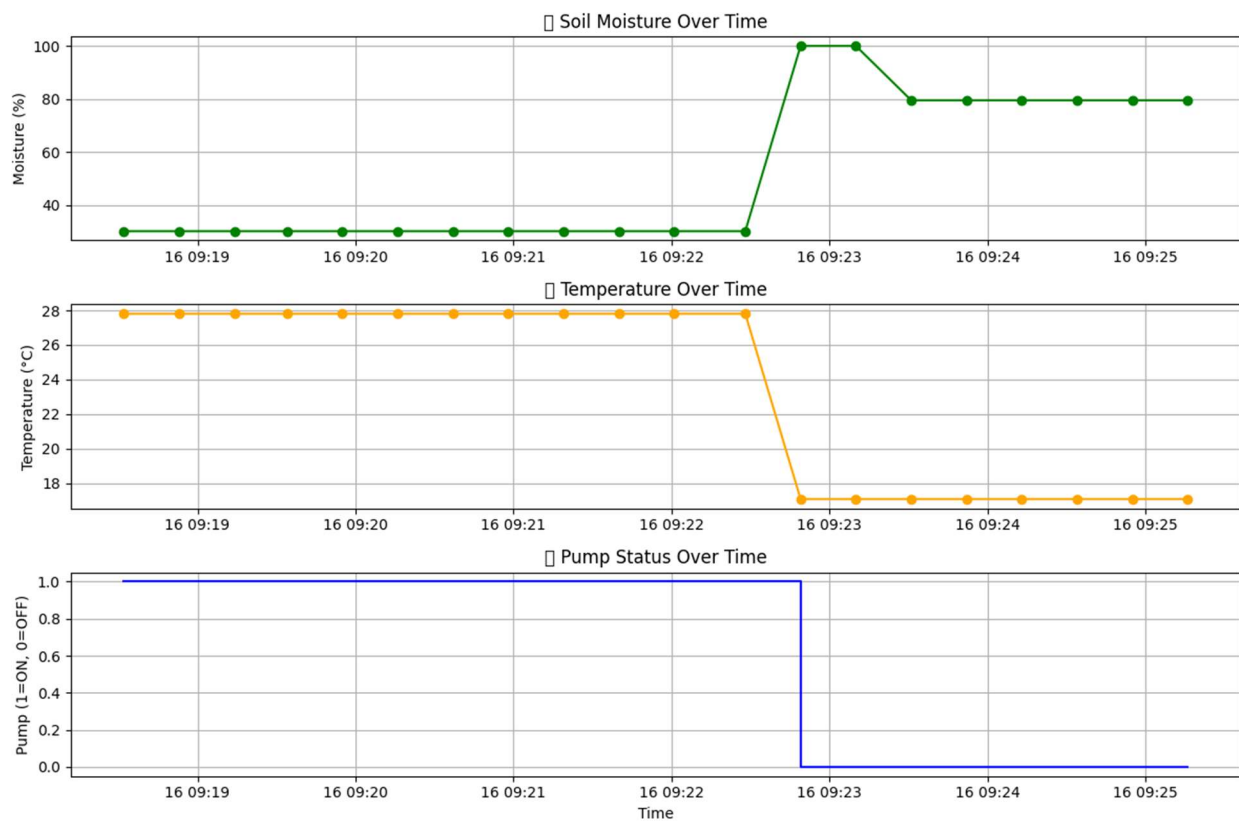
```
plt.xlabel('Time')
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

Output:



Github : <https://github.com/praveenkumar-30726/IIoT/blob/main/IIoT.ipynb>

The screenshot displays a Jupyter Notebook interface with a dark theme. The top navigation bar includes links for Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings. The main area shows a Python script for IoT data processing. The script imports necessary libraries, fetches data from a ThingSpeak API, converts it to a DataFrame, and displays the latest data. The script is as follows:

```
In [1]:  
# Import Required Libraries  
import pandas as pd  
import matplotlib.pyplot as plt  
import requests  
from datetime import datetime  
  
# ThingSpeak READ API URL  
url = "https://api.thingspeak.com/channels/3119564/feeds.json?api_key=CQSSV8GUL72W6P&results=20"  
  
# Fetch Data from ThingSpeak  
response = requests.get(url)  
data = response.json()  
  
# Convert to DataFrame  
feeds = data['feeds']  
df = pd.DataFrame(feeds)  
  
# Convert 'created_at' to readable datetime format  
df['created_at'] = pd.to_datetime(df['created_at'])  
  
# Extract fields (update field numbers if needed)  
df['Soil_Moisture'] = df['field1'].astype(float)  
df['Temperature'] = df['field2'].astype(float)  
df['Pump_Status'] = df['field3'].astype(float)  
  
# Display the Latest data  
print(df['created_at'], 'Soil_Moisture', 'Temperature', 'Pump_Status')[].tail()  
  
# Plot Graphs  
plt.figure(figsize=(12, 8))  
  
# --- Soil Moisture ---  
plt.subplot(3, 1, 1)  
plt.plot(df['created_at'], df['Soil_Moisture'], color='green', marker='o')  
plt.title('Soil Moisture Over Time')  
plt.xlabel('Moisture (X)')  
plt.grid(True)  
  
# --- Temperature ---  
plt.subplot(3, 1, 2)
```

Unity

Program:

```
using System.Collections;
```

```
using UnityEngine;
```

```
using UnityEngine.Networking;
```

```
using SimpleJSON;
```

```
public class IoTVisualizer : MonoBehaviour
```

```
{
```

```
    public GameObject soil;
```

```
    public GameObject plant;
```

```
    public GameObject pump;
```

```
    public Light sunLight;
```

```
    string thingSpeakURL =  
    "https://api.thingspeak.com/channels/3119564/feeds.json?api_key=CQASS9XGW1J2MWP  
    A&results=1";
```

```
    void Start()
```

```
    {
```

```
        StartCoroutine(UpdateDataLoop());
```

```
    }
```

```
IEnumerator UpdateDataLoop()
```

```
{
```

```
    while (true)
```

```
    {
```

```
        yield return GetThingSpeakData();
```

```
        yield return new WaitForSeconds(20f);
```

```
}  
}
```

IEnumerator GetThingSpeakData()

```
{  
    UnityWebRequest www = UnityWebRequest.Get(thingSpeakURL);  
    yield return www.SendWebRequest();
```

```
    if (www.result != UnityWebRequest.Result.Success)  
    {  
        Debug.LogError("ThingSpeak Error: " + www.error);  
        yield break;  
    }
```

```
    var json = JSON.Parse(www.downloadHandler.text);  
    var feeds = json["feeds"];  
    if (feeds.Count == 0) yield break;
```

```
    var latest = feeds[feeds.Count - 1];  
    float soilMoisture = latest["field1"].AsFloat;  
    float temperature = latest["field2"].AsFloat;  
    int pumpStatus = latest["field3"].AsInt;
```

```
    Debug.Log($" 🌱 Soil: {soilMoisture}% | 🌡 Temp: {temperature}°C | Pump:  
    {(pumpStatus == 1 ? "ON" : "OFF")}");
```

```
// === VISUALIZATION ===
```

```
UpdateSoilColor(soilMoisture);
```

```
UpdatePlant(soilMoisture);
```

```
UpdatePump(pumpStatus);
```

```
    UpdateLighting(temperature);  
}
```

```
void UpdateSoilColor(float moisture)
```

```
{  
    Renderer soilRenderer = soil.GetComponent<Renderer>();  
    if (moisture < 40)  
        soilRenderer.material.color = new Color(0.5f, 0.3f, 0.1f); // Dry brown  
    else  
        soilRenderer.material.color = new Color(0.2f, 0.1f, 0.05f); // Wet dark  
}
```

```
void UpdatePlant(float moisture)
```

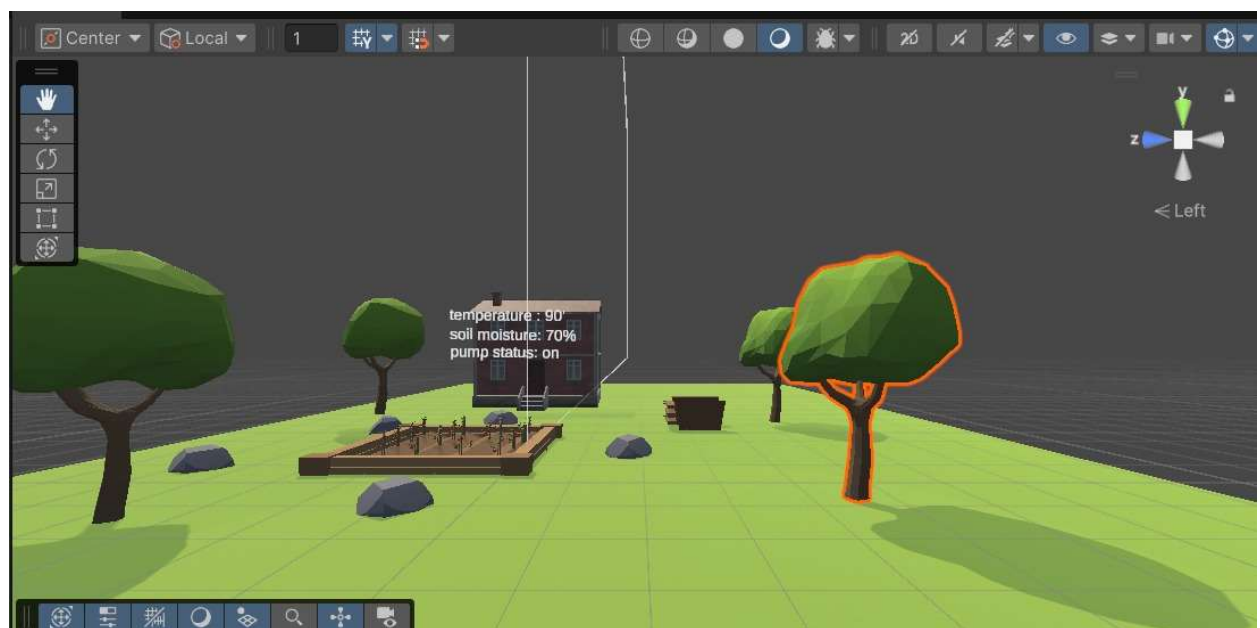
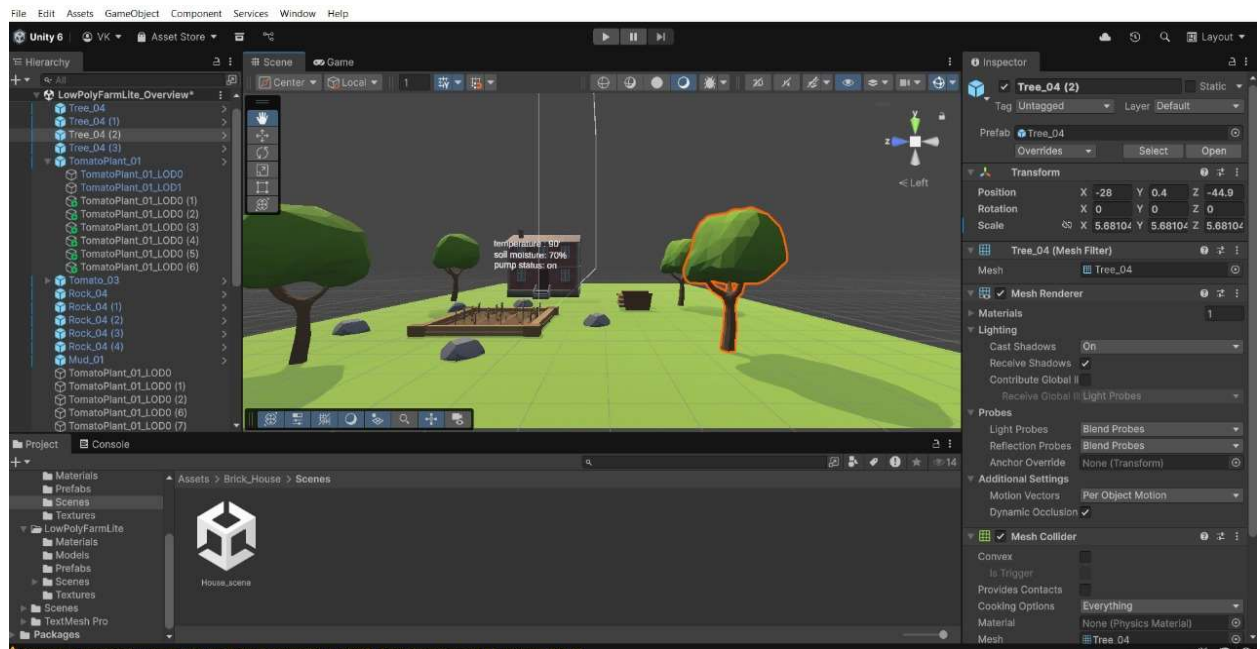
```
{  
    if (plant != null)  
    {  
        float scale = Mathf.Lerp(0.5f, 1.2f, moisture / 100f);  
        plant.transform.localScale = new Vector3(scale, scale, scale);  
    }  
}
```

```
void UpdatePump(int status)
```

```
{  
    if (pump != null)  
    {  
        pump.GetComponent<Renderer>().material.color = (status == 1) ? Color.blue :  
Color.gray;  
        if (status == 1)  
            pump.transform.Rotate(Vector3.up * 90 * Time.deltaTime);  
    }  
}
```

```
}

void UpdateLighting(float temperature)
{
    if (sunLight != null)
    {
        if (temperature > 35)
            sunLight.color = Color.red; // Hot
        else
            sunLight.color = Color.white; // Normal
    }
}
}
```



Future Scope of the Project

1. **AI and Machine Learning Integration:** To predict soil moisture, temperature, and control irrigation automatically.
2. **Advanced 3D Visualization:** Enhance Unity visuals with weather effects, water flow, and realistic plant growth animations.
3. **Mobile & VR Support:** Enable remote farm monitoring using mobile apps or virtual reality devices.
4. **Cloud-Based Analytics:** Use ThingSpeak, Google Colab, and Power BI for data storage, analysis, and visualization.
5. **Automation & Smart Control:** Fully automate pump operations based on sensor data and AI predictions.
6. **Voice & Remote Access:** Integrate with Google Assistant or Alexa for hands-free control.
7. **Scalable IoT Network:** Expand the system to monitor multiple fields or sensors under one dashboard.
8. **Data Reporting & Insights:** Generate reports and long-term analytics for performance tracking.
9. **Energy Efficiency:** Incorporate solar power or low-power IoT modules for sustainable operation.
10. **Security Enhancement:** Add data encryption and authentication to protect IoT communications.